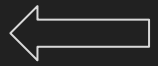
Rust for Embedded

Embedded Systems 2025/2026

Outline

- Why learning Rust? 
- Getting started with Rust
  - Setting up a development environment
  - Hello, world!
  - Rust projects: cargo
- Rust for embedded (STM32 example)
  - Bare metal programming
  - HAL based project

References:

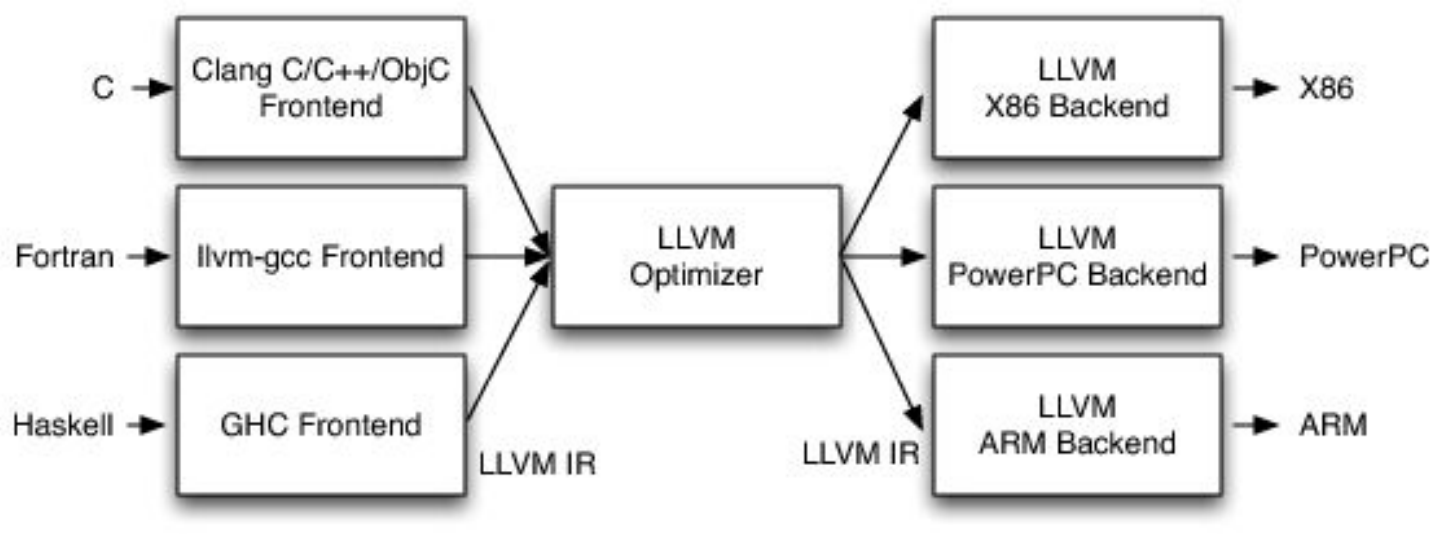
- Official Rust book: https://rust-book.cs.brown.edu/
- Rust embedded book: https://docs.rust-embedded.org/book/
- Rust Training all in one: https://tyrchen.github.io/rust-training/rust-training-all-in-one.html
- Example repository: https://github.com/alarmfox/rust4embedded

Rust

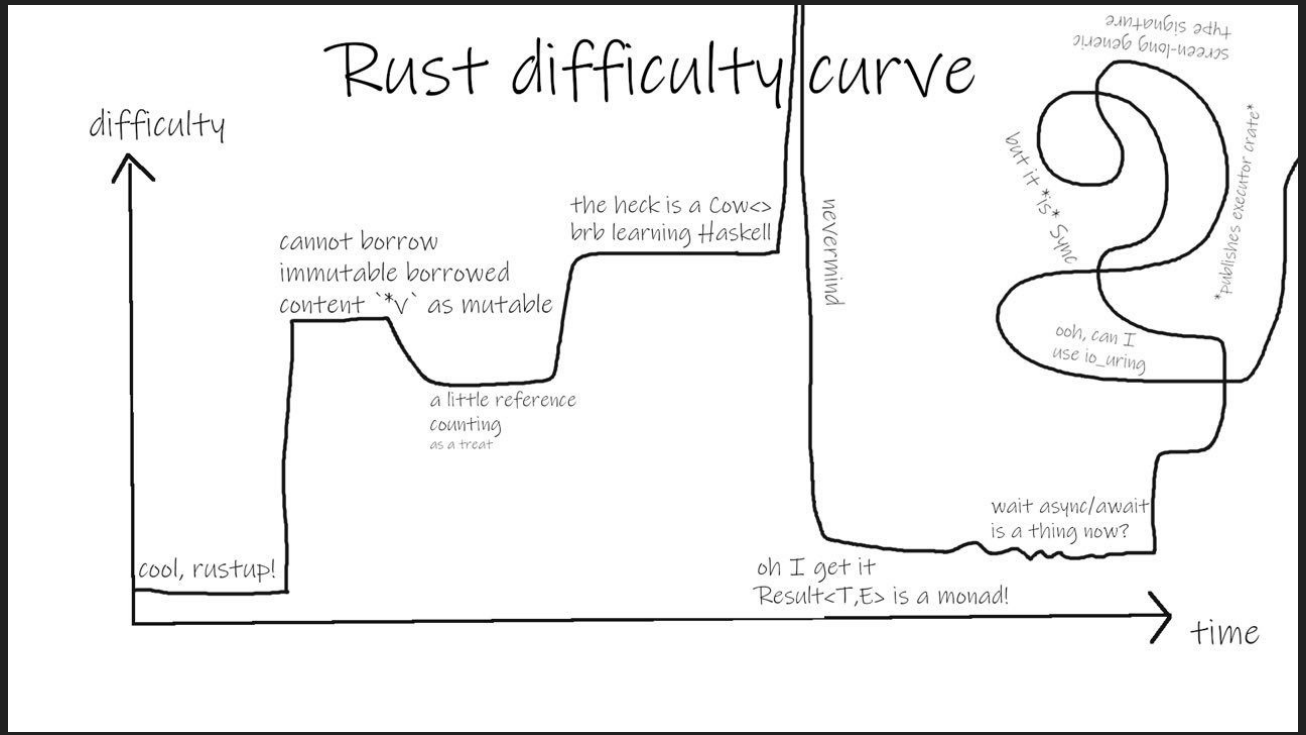
- Rust is a compiled language developed with **memory safety, type safety** and **performance** in mind
- Combines different programming paradigms:
  - Imperative
  - Functional
  - OOP-like
- Rust removes different vulnerability classes such as **double free**, **overflow** and **use after free** enforcing checks at compile time
- Enforces memory management at compile time using an *ownership* model with the **borrow checker**

Rust

- Rust is an LLVM compiled language (LLVM: low-level virtual machine)
  - Frontend: compile language to an IR (intermediate representation)
  - Backend: translate IR to architecture dependent instructions ISA

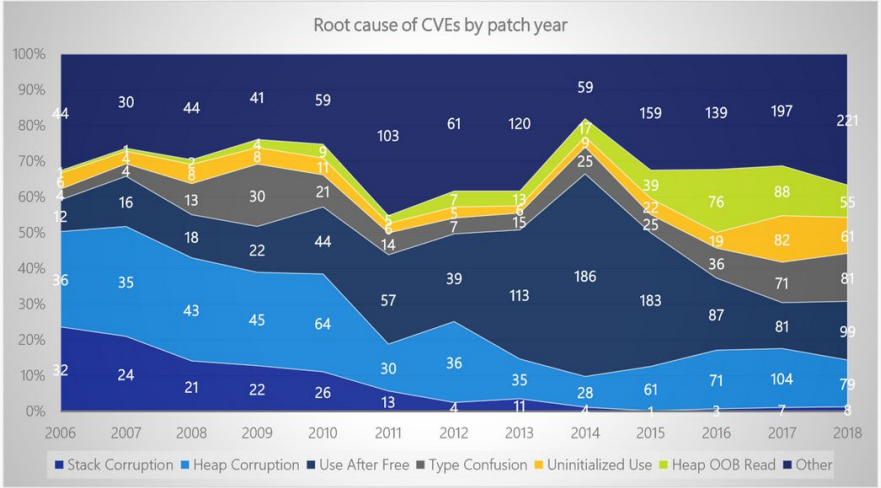


Rust: not a simple language



Rust: why

Drilling down into root causes



Stack corruptions are essentially dead

Use after free spiked in 2013-2015 due to web browser UAF, but was mitigated by Mem GC

Heap out-of-bounds read, type confusion, & uninitialized use have generally increased

Spatial safety remains the most common vulnerability category (heap out-of-bounds read/write)

Top root causes since 2016:

- #1: heap out-of-bounds
- #2: use after free
- #3: type confusion
- #4: uninitialized use

11

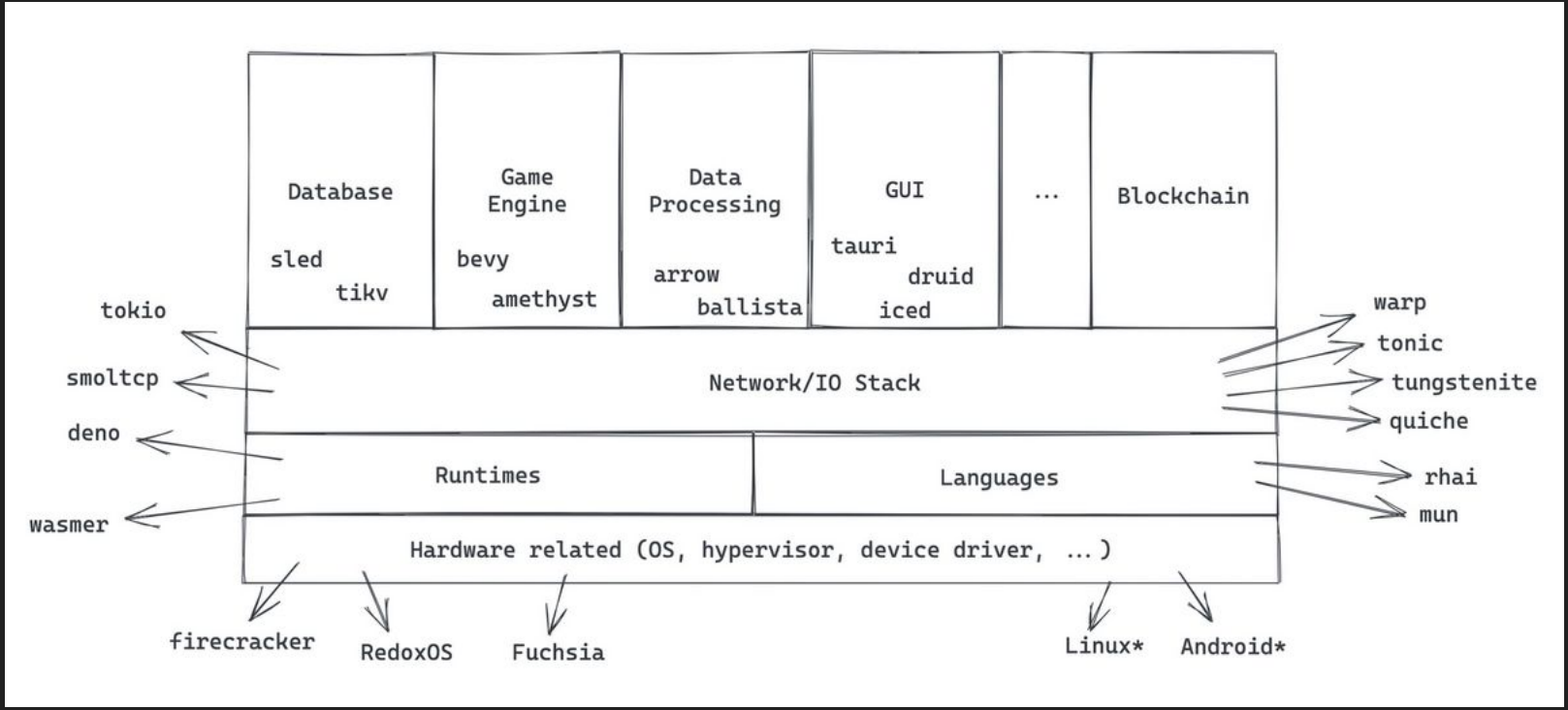
Note: CVEs may have multiple root causes, so they can be counted in multiple categories

https://www.grammatech.com/learn/memory-safety-issues-are-still-the-leading-source-of-security-vulnerabilities/

Rust in the real-world

- High performance services
  - Cloudflare Pingora serves up to a 40 million requests per second with 30% resource usage compared to Nginx https://blog.cloudflare.com/pingora-open-source/
  - rust-vmm powers major cloud hypervisors (based on KVM or Microsoft Hypervisor) like Firecracker https://github.com/firecracker-microvm/firecracker
  - Microsoft, Google and Amazon are using Rust in their kernels/products (Windows, Fuchsia, Android, AWS-Lambda) https://blog.cloudflare.com/pingora-open-source/, https://github.com/rust-vmm
  - Rust is also in the Linux kernel https://docs.kernel.org/rust/index.html
- Embedded systems projects
  - IBM ACE: a formal verified firmware for TEE on RISC-V https://github.com/IBM/ACE-RISCV
  - TockOS: embedded OS for Cortex-M and RISC-V IoT devices https://book.tockos.org/
  - OpenSK: Google implementation to manage security keys for FIDO2 https://github.com/google/opensk

Rust in the real-world



Unsafe code: buffer overflow

- *gets* is **unsafe**
  - No checks on the input size
  - Writing more than 8 characters makes the program crash
- Understanding the memory layout:
  - For alignment reasons, GCC puts 8-bytes **passwords** before **is_admin**
- We could reach *is_admin* and get access

```c
int main() {
    /* Memory layout: 'is_admin' is often placed right next to 'password' on the stack */
    int is_admin = 0;
    char password[8];

    printf("Enter password: ");
    /* VULNERABILITY: gets() doesn't check the length of the input! */
    gets(password);

    if (strcmp(password, "secret") == 0) {
        is_admin = 1;
    }

    if (is_admin) {
        printf("Access Granted! Admin privileges unlocked.\n");
    } else {
        printf("Access Denied.\n");
    }
    return 0;
}
```

Unsafe code: buffer overflow

- There are techniques to exploit more complex scenario and understand memory layout:
  - De Bruijn sequence
  - Automated Python tools
- Countermeasures:
  - Stack protector
  - Stack randomization (makes it harder, but not impossible)
  - A **safe** language

```c
int main() {
    /* Memory layout: 'is_admin' is often placed right next to 'password' on the stack */
    int is_admin = 0;
    char password[8];

    printf("Enter password: ");
    /* VULNERABILITY: gets() doesn't check the length of the input! */
    gets(password);

    if (strcmp(password, "secret") == 0) {
        is_admin = 1;
    }

    if (is_admin) {
        printf("Access Granted! Admin privileges unlocked.\n");
    } else {
        printf("Access Denied.\n");
    }
    return 0;
}
```

Unsafe code: buffer overflow

- Rust String type is heap allocated and dynamically sized
- Even if we put a very-big input the program is simply going to crash

```rust
fn main() {
    let mut is_admin = false;
    // Rust's String is dynamically sized and heap-allocated
    let mut password = String::new();

    println!("Enter password: ");
    // read_line automatically grows the String to fit the input safely
    io::stdin().read_line(&mut password).unwrap();

    if password.trim() == "secret" {
        is_admin = true;
    }

    if is_admin {
        println!("Access Granted! Admin privileges unlocked.");
    } else {
        println!("Access Denied.");
    }
}
```

Unsafe code: use after free

- We should mark the pointer NULL after the *free*
- If we trigger an allocation right after the *free*, we can write to the data previously used by the *user* struct
  - This can be done using timing attacks
- This seems very easy, right?
  - https://www.cve.news/cve-2026-5281/
  - https://whiteknightlabs.com/2025/06/03/understanding-use-after-free-uaf-in-windows-kernel-drivers/

```c
void regular_greet() {
    printf("Hello, user! Welcome back.\n");
}

struct User {
    char name[32];
    void (*action)(); // Function pointer
};

int main() {
    struct User *user = malloc(sizeof(struct User));
    strcpy(user->name, "Alice");
    user->action = regular_greet;

    user->action();

    free(user);

    /* For simplicity this is placed here. But the allocation can be
     * triggered in another part of the program (far away) */
    long *attacker_data = malloc(sizeof(struct User));

    attacker_data[0] = 0x41414141; // 'AAAA'
    attacker_data[1] = 0x41414141; // 'AAAA'
    attacker_data[2] = 0x41414141; // 'AAAA'
    attacker_data[3] = 0x41414141; // 'AAAA'
    attacker_data[4] = (long)drop_shell; // OVERWRITE THE FUNCTION POINTER!

    /* The program uses the same pointer to call action after calling free */
    printf("Executing user action...\n");
    user->action(); // The program blindly jumps to drop_shell!

    return 0;
}
```

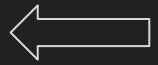
Unsafe code: use after free

- This Rust code simply does not compile
  - Drop **takes ownership** of user variable, it cannot be used afterwards

```
 > rustc uaf.rs
error[E0382]: use of moved value: `user`
  --> uaf.rs:27:5
   |
17 |     let user = User {
   |         ---- move occurs because `user` has type `User`, which does not implement the `Copy` trait
...
24 |     drop(user);
   |          ---- value moved here
...
27 |     (user.action)();
   |     ^^^^^^^^^^^^^^ value used here after move
   |
note: if `User` implemented `Clone`, you could clone the value
  --> uaf.rs:1:1
   |
 1 | struct User {
   | ^^^^^^^^^^^ consider implementing `Clone` for this type
...
24 |     drop(user);
   |          ---- you could clone this value

error: aborting due to 1 previous error
```

```rust
struct User {
    name: String,
    action: fn(),
}

fn regular_greet() {
    println!("Hello, user! Welcome back.");
}

fn drop_shell() {
    println!("[!] Hijacked execution flow! Dropping shell...");
    // Rust can call system commands via std::process::Command,
    // but we won't even get that far.
}

fn main() {
    let user = User {
        name: String::from("Alice"),
        action: regular_greet,
    };

    // We explicitly free the memory by dropping the variable.
    // In Rust, `drop` takes ownership of `user`.
    drop(user);

    // Attempt to invoke the function pointer after the memory is freed.
    (user.action)();
}
```

Outline

- Why learning Rust?
- Getting started with Rust 
  - Setting up a development environment
  - Hello, world!
  - Rust projects: cargo
- Rust for embedded (STM32 example)
  - Bare metal programming
  - HAL based project

References:

- Official Rust book: https://rust-book.cs.brown.edu/
- Rust embedded book: https://docs.rust-embedded.org/book/
- Rust Training all in one: https://tyrchen.github.io/rust-training/rust-training-all-in-one.html
- Example repository: https://git.capass.org/rust4embedded/

Rust: getting started

- Rust is a modern programming language: *https://rust-lang.org/tools/install/*
  - Modern installation procedure
  - Comes with package and project manager: *cargo*
  - Includes development tools: linter, formatter, LSP
    - rustup component add rust analyzer clippy
- It achieves high performance memory management (and safety) at compile time
  - Immutability by default
  - Borrow checker
- Rust is not magic: we can *still* make a lot of damage
  - Safe Rust vs Unsafe Rust

Rust: variables, lifetime and ownership model

- Standard types imply their size:
  - i8, u8, i32, f64 ecc
  - Type inferred is usually correct
  - Specific type
- Two types for strings:
  - &str: size known at compile time
  - String: heap allocated
- Use *rust_analyzer* for a good DX

```rust
fn main() {
    let a: &str = "Hello world";
    let n: i32 = 10;

    println!("{} {}", a, n); // Prints hello world and 10

    /* We can rename variables. The one with the closer scope wins */
    let n: String = String::from(a);

    /* The type of copy will be automatically inferred by the the rust compiler (String) */
    let copy = n.clone();

    /* function ending in ! are not functions but macros. Rust has a meta-language */
    println!("{} {}", a, n); // Prints hello world twice
}
```

Rust: variables, lifetime and ownership model

- Variable lifetimes are assigned based on the *scope*
- *let* declares a variable
  - Rust has type inference
  - Variable are by default read-only
- Use *mut* to make them mutable

```rust
fn main() {
    /* Scope 1*/
    let b = 5;

    /* Ok */
    let c = {
        let a = 10;

        /* A block is an expression: it can produce a value
         * a + b is the result of this expression and is assigned to C
         *
         * This is ok because b leaves in an outer scope and a is local scoped.
         */
        a + b
    };
    println!("{}", c);

    /* Not ok: a is deallocated when its scope ends. This code does not compile */
    let mut c = a + b;
    c += 1;

    println!("{}", c);
}
```

Rust: variables, lifetime and ownership model

- A variable is the owner of a memory area
- One owner per memory area is enforced by the rust compiler
- We can *borrow* the ownership by taking a reference (&)
- This solves the *double free* error
  - If we would allow s2 and s1, Rust would deallocate both by freeing the same memory area

```rust
fn main() {
    // 1. s1 is created and becomes the owner of the String "hello"
    let s1 = String::from("hello");

    // 2. We assign s1 to s2.
    // In Rust, this means ownership MOVES from s1 to s2.
    // s1 is now considered empty/invalid.
    let s2 = s1;

    // 3. If we try to print s1, Rust will throw a COMPILE ERROR!
    // println!("{}, world!", s1);

    // 4. We can only print s2, because s2 is the new owner.
    println!("{}, world!", s2);
}
```

```rust
fn main() {
    // 1. s1 is created and becomes the owner of the String "hello"
    let s1 = String::from("hello");

    // 2. Borrow s1. We take a read-only reference. s1 is still the owner
    let s2 = &s1;

    // or we can clone and create 2 distinct memory areas. s1 and s2 are both valid
    let s2 = s1.clone();

    // 3. Use the reference
    println!("{}, world!", s1);

    // 4. We can only print s2, because s2 is the new owner.
    println!("{}, world!", s2);
}
```

Rust: ownership model in detail

- *take_and_modify* takes ownership of the variable *s*
- Calling from main, we lose *my_text* ownership
- We need to make *take_and_modify* to return the string to be able to access it again with a new *variable*

```rust
fn main() {
    let my_text = String::from("Hello");

    // We pass 'my_text' into the function. Ownership MOVES into the function.
    let returned_text = take_and_modify(my_text);

    // println!("{}", my_text); // COMPILE ERROR! my_text no longer owns the data.

    // We must use the returned variable because it is the new owner.
    println!("Final result: {}", returned_text);
}

// This function takes ownership of 's'. We make 's' mutable so we can change it.
fn take_and_modify(mut s: String) -> String {
    s.push_str(" World"); // Modifies the string

    s // We MUST return 's' to give ownership back to the caller!
}
```

Rust: ownership model in detail

- *take_and_modify* takes *a mutable reference* of the variable *s* (borrowing)
- Calling from main, we can create a temporary *mutable reference*
- There can only be one mutable reference active per time

```rust
fn main() {
    // We must declare the original variable as mutable (mut)
    let mut my_text = String::from("Hello");

    // We pass a MUTABLE REFERENCE to the function using '&mut'
    // Ownership does NOT move. The function is just borrowing it.
    modify_in_place(&mut my_text);

    // This works perfectly! We never lost ownership of my_text.
    println!("Final result: {}", my_text);
}

// This function takes a mutable reference to a String.
// It does not take ownership, and it doesn't need to return anything.
fn modify_in_place(s: &mut String) {
    s.push_str(" World"); // Modifies the original string directly
}
```

```rust
fn main() {
    // We must declare the original variable as mutable (mut)
    let mut my_text = String::from("Hello");

    // Create another temporary mutable reference
    let mut temp_ref = &mut my_text;

    // We pass a MUTABLE REFERENCE to the function using '&mut'
    // Ownership does NOT move. The function is just borrowing it.
    modify_in_place(&mut my_text);

    // Use the temporary reference created before here.
    // This gives an error: we have 2 mutable reference active
    temp_ref.push_str("aaaa");

    // This works perfectly! We never lost ownership of my_text.
    println!("Final result: {}", my_text);
}

// This function takes a mutable reference to a String.
// It does not take ownership, and it doesn't need to return anything.
fn modify_in_place(s: &mut String) {
    s.push_str(" World"); // Modifies the original string directly
}
```

Rust safety: enums

- Enums are *sum* types
  - Enums have variant
- Standard enums: Result<T, E> and Option<T>
  - Error as values in a modern language
  - Avoid Null pointer dereference
- *match* statement for exhaustive pattern matching

```rust
// This enum has four variants.
// It is a "Sum Type" because a WebEvent is exactly ONE of these at a time.
enum WebEvent {
    PageLoad,                // Variant 1: No data attached
    KeyPress(char),          // Variant 2: Holds a single character
    Click { x: i64, y: i64 }, // Variant 3: Holds an anonymous struct
}

fn handle_event(event: WebEvent) {
    // 3. Use 'match' to exhaustively check every possible variant.
    // Notice how we give variable names like 'c', 'x', and 'y' to pull
    // the data out of the variants right when we match them!
    match event {
        WebEvent::PageLoad => {
            println!("Action: The page has completely finished loading.");
        }
        WebEvent::KeyPress(c) => {
            println!("Action: The user pressed the '{}' key.", c);
        }
        WebEvent::Click { x, y } => {
            println!(
                "Action: The mouse was clicked at coordinates x={}, y={}.",
                x, y
            );
        }
    }
}

fn main() {
    // We instantiate different variants of the same enum type
    let event1 = WebEvent::PageLoad;
    let event2 = WebEvent::KeyPress('A');
    let event3 = WebEvent::Click { x: 100, y: 200 };

    handle_event(event1);
    handle_event(event2);
    handle_event(event3);
}
```

Rust safety: enums

- Enums are *sum* types
  - Enums have variant
- Standard enums: Result<T, E> and Option<T>
  - Error as values in a modern language
  - Avoid Null pointer dereference
- *match* statement for exhaustive pattern matching

```rust
// This function might find a user, or it might not.
// It returns an Option, forcing the caller to handle the 'None' case.
fn find_user_name(id: u32) -> Option<String> {
    if id == 1 {
        Some(String::from("Alice")) // We found a value!
    } else {
        None // We didn't find a value. No null pointers here!
    }
}

fn main() {
    let user = find_user_name(2);

    // EXHAUSTIVE PATTERN MATCHING:
    // We use 'match' to check the variants. Rust will refuse to compile
    // if we forget to handle the 'None' case! This prevents crashes.
    match user {
        Some(name) => println!("Found user: {}", name),
        None => println!("User not found. Please try again."),
    }
}
```

Rust safety: enums

- Enums are *sum* types
  - Enums have variant
- Standard enums: Result<T, E> and Option<T>
  - Error as values in a modern language
  - Avoid Null pointer dereference
- *match* statement for exhaustive pattern matching

```rust
// This function returns a Result.
// T (Success) is an f64. E (Error) is a String.
fn divide(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("Cannot divide by zero!")) // Returning the error as a value
    } else {
        Ok(a / b) // Returning the success value
    }
}

fn main() {
    let outcome = divide(10.0, 0.0);

    // EXHAUSTIVE PATTERN MATCHING:
    // Rust forces us to handle both the Ok and Err variants.
    // You cannot accidentally use the result of a failed calculation.
    match outcome {
        Ok(number) => println!("The result is {}", number),
        Err(error_msg) => println!("Calculation failed: {}", error_msg),
    }
}
```

Rust: struct

- Struct define a custom type
- Implement functions on structs
- Implement and define *trait* for common behaviour
  - Trait are similar to interfaces
- Rust has a codegen feature
  - Implement Debug and Clone trait by just declaring it

```rust
/* Derive we are asking to the compiler to generate some code for us. For example, Debug and Clone
 * functions for this struct. The implementations will call recursively the same function for each
 * member */
#[derive(Debug, Clone)]
struct Player {
    name: String,
    age: u8,
    alive: bool,
}

/* Add operations on struct with "impl" blocks. Impl blocks can add simple operations or
 * implement a "trait" (like interfaces) */
impl Player {
    /* By default everything is private */
    fn hi(&self) {
        /* In rust everything is an expression that can generate a value.
         * Expressions MUST generate value of the same type */
        let str = if self.alive {
            /* Note no ";" */
            format!("{} - {}, alive", self.name, self.age)
        } else {
            /* Note no ";" */
            format!("{} - {}, dead", self.name, self.age)
        };

        println!("{}", str);
    }
}

/* Implement for example the Default trait */
impl Default for Player {
    fn default() -> Self {
        Self {
            name: Default::default(),
            age: 18,
            alive: true,
        }
    }
}
```

Rust projects: cargo

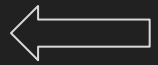
- *cargo init*: to create a new project
- *cargo run*: build and run
- *Cargo.toml*: dependencies
- *cargo test*: run test cases

```
> tree .
.
├── Cargo.lock
├── Cargo.toml
└── src
    └── main.rs
```

```toml
[package]
name = "password-manager"
version = "0.1.0"
edition = "2024"

[dependencies]
argon2 = "0.5.3"
clap = { version = "4.5", features = ["derive"] }
chacha20poly1305 = "0.10.1"
hex = "0.4.3"
rand_core = { version = "0.6.4", features = ["getrandom"] }
rpassword = "7.4.0"
```

- *Let's have a look at the password-manager*
  - Use cryptography functions
  - Manage structures and enums

Outline

- Why learning Rust?
- Getting started with Rust
  - Setting up a development environment
  - Hello, world!
  - Rust projects: cargo
- Rust for embedded (STM32 example) 
  - Bare metal programming
  - HAL based project

References:

- Official Rust book: https://rust-book.cs.brown.edu/
- Rust embedded book: https://docs.rust-embedded.org/book/
- Rust Training all in one: https://tyrchen.github.io/rust-training/rust-training-all-in-one.html
- Example repository: https://git.capass.org/rust4embedded/

Setting up a bare metal project (STM32F303VC-discovery)

- First install the target: *rustup target add thumbv7em-none-eabihf*
- Create a new project and configure it using the *.cargo/config.toml*

```
[build]
# Specify the default target. This can be overridden by passing the "--target " flag to cargo
target = "thumbv7em-none-eabihf"

# Pass some arguments to the linker
rustflags = [
  "-C", "link-arg=-Tmemory.x",

  # if you run into problems with LLD switch to the GNU linker by commenting out
  # this line
  # "-C", "linker=arm-none-eabi-ld",
]
```

Setting up a bare metal project (STM32F303VC-discovery)

- Create a simple memory layout: for now just put everything in RAM
- Put the *_init* symbol at the beginning of .text so that we can initialize it

```
MEMORY
{
  /* Datasheet */
  RAM : ORIGIN = 0x20000000, LENGTH = 40K
}

/* We need a 128B of stack to start with. Place it at the end of memory */
_stack_size = 128;
_stack_top  = ORIGIN(RAM) + LENGTH(RAM);

SECTIONS {

  /* text region */
  .text : ALIGN(4K) {
    KEEP(*(._init));
    . = ALIGN(4K);
    *(.text .text.*);
  } > RAM
}
```

Setting up a bare metal project (STM32F303VC-discovery)

- *_init* initializes the sp, zero out registers and jumps to main
- _init needs to be *naked* -> remove the function prologue and epilogue
- *no_mangle* tells the rust compiler to not change the name of the function
  - The linker will exactly for the _init symbol when creating the ELF

```rust
#[unsafe(naked)]
#[unsafe(no_mangle)]
extern "C" fn _init() -> ! {
    core::arch::naked_asm!(
        // 1. Setup Stack Pointer
        // We use the LLVM assembler pseudo-instruction '=' to load the address
        // of the external linker symbol `_stack_top` into r0, then move it to sp.
        "ldr r0, =_stack_top",
        "mov sp, r0",

        // 2. Clear / Reset general-purpose registers (r0-r12)
        // Loading 0 into r0, then moving r0 into the rest is the most efficient way.
        "movs r0, #0",
        "mov r1, r0",
        "mov r2, r0",
        "mov r3, r0",
        "mov r4, r0",
        "mov r5, r0",
        "mov r6, r0",
        "mov r7, r0",
        "mov r8, r0",
        "mov r9, r0",
        "mov r10, r0",
        "mov r11, r0",
        "mov r12, r0",

        // Clear the Link Register (lr). This is helpful for debuggers so they
        // know they have reached the bottom of the call stack.
        "mov lr, r0",

        // 3. Jump to main
        // We use Rust's `sym` to resolve the address of the main function safely
        "b {main}",
        main = sym main,
    )
} fn _init
```

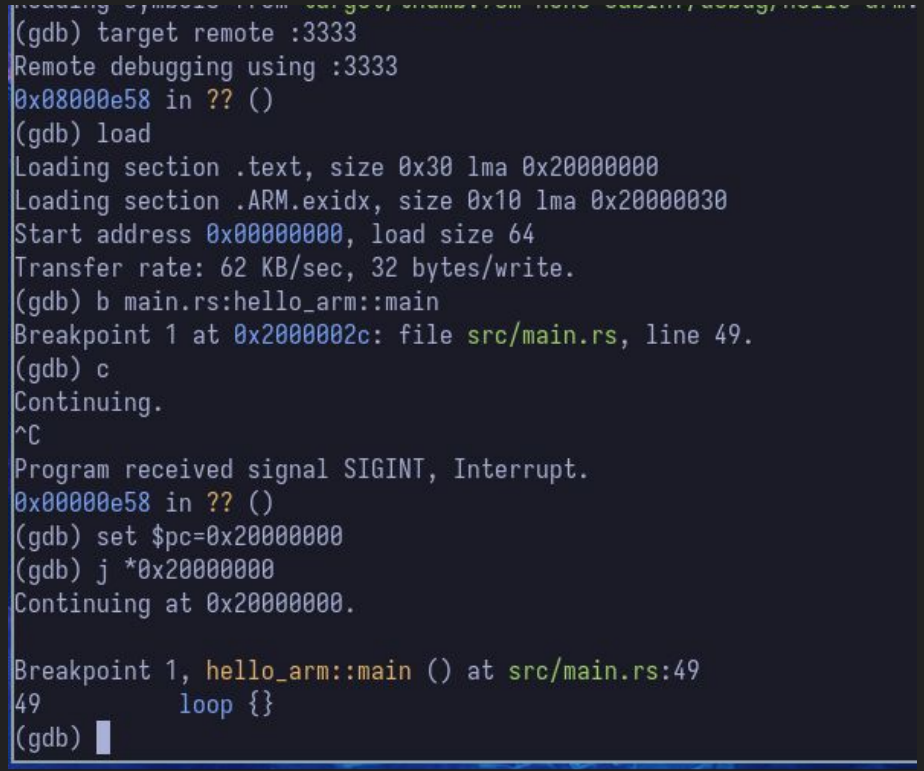
Setting up a bare metal project (STM32F303VC-discovery)

- Create a *build.rs* in the root project directory
  - A Rust program that runs before the build
- We can emit special instructions to the rust compiler or perform certain operations

```rust
fn main() {
    // Put the linker script somewhere the linker can find it
    let out = &PathBuf::from(env::var_os("OUT_DIR").unwrap());
    File::create(out.join("memory.x"))
        .unwrap()
        .write_all(include_bytes!("memory.x"))
        .unwrap();
    println!("cargo:rustc-link-search={}", out.display());

    // Only re-run the build script when memory.x is changed,
    // instead of when any part of the source code changes.
    println!("cargo:rerun-if-changed=memory.x");
}
```

Setting up a bare metal project (STM32F303VC-discovery)

- Build with *cargo build*
- Program the device using the *arm-none-eabi-gdb*
  - Don't forget to start *openocd* after connecting the device to the host PC
  - *load* the ELF
  - Put a breakpoint in main
  - Since we do not specify ENTRY in *memory.x*, we need to manually set the PC



Integrate the vector table (STM32F303VC-discovery)

- Integrate the vector table
  - Automatically populate sp and jump to reset
- Use semihosting to print a message to the serial
  - Remember to enable it with *monitor arm semihosting enable*
- Toggle a LED using raw addresses

```rust
// Import the stack top symbol from our linker script
unsafe extern "C" {
    static _stack_top: u32;
}

// Define the structure of the bare minimum Vector Table
//  GDB look at the vector table x/2xw 0x08000000
#[repr(C)]
struct VectorTable {
    initial_sp: &'static u32,
    reset_handler: unsafe extern "C" fn() -> !,
}

// Create the vector table and force it into the ".vector_table" section
#[unsafe(link_section = ".vector_table")]
#[unsafe(no_mangle)]
static VECTOR_TABLE: VectorTable = VectorTable {
    // We take the address of the linker symbol to give the hardware the stack top
    initial_sp: unsafe { &_stack_top },
    // We point the hardware directly to our initialization function
    reset_handler: _init,
};
```

Integrate the vector table (STM32F303VC-discovery)

- A simple LED struct
- Enable the GPIOE using raw bit shifting operations
- Use always *volatile* to avoid caching issues

```rust
// A struct to represent a single LED on Port E
pub struct Led {
    pin: u8,
}

impl Led {
    /// Initializes Port E and configures the specific pin as an output.
    pub fn new(pin: u8) -> Self {
        unsafe {
            // 1. Enable the clock for GPIO Port E
            let mut ahbenr = core::ptr::read_volatile(RCC_AHBENR);
            ahbenr |= 1 << 21;
            core::ptr::write_volatile(RCC_AHBENR, ahbenr);

            // 2. Configure the specific pin as an output (01)
            let mut moder = core::ptr::read_volatile(GPIOE_MODER);
            let shift = pin * 2; // Each pin takes 2 bits in MODER
            moder &= !(0b11 << shift); // Clear bits
            moder |= 0b01 << shift; // Set to output
            core::ptr::write_volatile(GPIOE_MODER, moder);
        }

        // Return the constructed Led struct
        Self { pin }
    }

    /// Toggles the LED on and off. Notice this function is completely SAFE!
    pub fn toggle(&self) {
        unsafe {
            let mut odr = core::ptr::read_volatile(GPIOE_ODR);
            odr ^= 1 << self.pin;
            core::ptr::write_volatile(GPIOE_ODR, odr);
        }
    }
}
```

Integrate the vector table (STM32F303VC-discovery)

- Use *bkpt 0xAB* to write a character on the serial
- Implement the *Write* trait to get all print functionalities

```rust
#[unsafe(no_mangle)]
fn main() -> ! {
    let mut serial: HostSerial = serial::HostSerial;
    let a: i32 = 2 + 2;

    // Use the standard core::fmt macro to print strings and formatted numbers
    let _ = writeln!(serial, "Hello from bare-metal Rust! 2 + 2 = {}", a);
```

```rust
use core::arch::asm;
use core::fmt::{self, Write};

/// An empty struct we will use to implement the Write trait
pub struct HostSerial;

impl Write for HostSerial {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        // The semihosting operation code for printing a single character
        const SYS_WRITEC: usize = 0x03;

        for byte: u8 in s.bytes() {
            unsafe {
                // r0 holds the semihosting command
                // r1 holds a POINTER to the character we want to print
                asm!(
                    "bkpt 0xAB",
                    inout("r0") SYS_WRITEC => _,
                    in("r1") core::ptr::addr_of!(byte),
                    options(nostack, preserves_flags)
                );
            }
        }
        Ok(())
    }
}
```

Integrate the vector table (STM32F303VC-discovery)

- Use *bkpt 0xAB* to write a character on the serial
- Implement the *Write* trait to get all print functionalities

```rust
#[unsafe(no_mangle)]
fn main() -> ! {
    let mut serial: HostSerial = serial::HostSerial;
    let a: i32 = 2 + 2;

    // Use the standard core::fmt macro to print strings and formatted numbers
    let _ = writeln!(serial, "Hello from bare-metal Rust! 2 + 2 = {}", a);
```

```rust
use core::arch::asm;
use core::fmt::{self, Write};

/// An empty struct we will use to implement the Write trait
pub struct HostSerial;

impl Write for HostSerial {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        // The semihosting operation code for printing a single character
        const SYS_WRITEC: usize = 0x03;

        for byte: u8 in s.bytes() {
            unsafe {
                // r0 holds the semihosting command
                // r1 holds a POINTER to the character we want to print
                asm!(
                    "bkpt 0xAB",
                    inout("r0") SYS_WRITEC => _,
                    in("r1") core::ptr::addr_of!(byte),
                    options(nostack, preserves_flags)
                );
            }
        }
        Ok(())
    }
}
```

Setting up a Rust embedded project: the PRO way

- Leverage the app-template: https://github.com/knurling-rs/app-template/
- First install *cargo-generate*: *cargo install cargo-generate*
- Install *probe-rs*: *https://probe.rs/docs/getting-started/installation/*
  - Used instead of OpenOCD to detect connected boards
- Install *flip-link*: *cargo install flip-link*
- Create the project with: *cargo generate --git https://github.com/knurling-rs/app-template --branch main --name my-app*
- Find the HAL for your board
  - For example for STM32F3xx boards https://github.com/stm32-rs/stm32f3xx-hal/

Setting up a Rust embedded project: the PRO way

- Configure the project addressing the TODOs
  - Choose the target *.config/cargo.toml (*choose from the proposed)
  - Choose the board *probe-rs chip list* and populate the “runner” property
  - Add the HAL library in *Cargo.toml (*add also *critical-section* to run the example)

```
[dependencies]
cortex-m = { version = "0.7", features = ["critical-section-single-core"] }
cortex-m-rt = "0.7"
defmt = "1.0"
defmt-rtt = "1.0"
panic-probe = { version = "1.0", features = ["print-defmt"] }
semihosting = "0.1.20"
# TODO(4) enter your HAL here
stm32f3xx-hal = { version = "0.10.0", features = ["ld", "rt", "stm32f303xc"] }
critical-section = "1.1.2"
```

Setting up a Rust embedded project: the PRO way

- Test everything with: *cargo test - - lib*
- Have a look at the demo project: *bin/demo.rs*

```
> cargo test --lib
   Compiling cortex-m-semihosting v0.5.0
   Compiling defmt-test-macros v0.3.2
   Compiling defmt v0.3.100
   Compiling defmt-test v0.3.3
   Compiling hello-arm-hal v0.1.0 (/home/giuseppe/dev/work/rust4embedded/hello-arm-hal)
    Finished `test` profile [optimized + debuginfo] target(s) in 0.47s
     Running unittests src/lib.rs (target/thumbv7em-none-eabihf/debug/deps/hello_arm_hal-86ac74ad82c39b21)
      Erasing ✔100% [####################]   8.00 KiB @  38.07 KiB/s (took 0s)
  Programming ✔100% [####################]   8.00 KiB @  17.78 KiB/s (took 0s)
     Finished in 0.76s
(1/1) running `it_works`...
all tests passed!
Firmware exited successfully
```

Thank you